

Project Testing Phase

Performance Testing Document

Date	30 July 2026	Team ID	Individual / Solo Project
Student	Bandhanadam Antony	Project Name	Automated Network Request Management in ServiceNow

1. Purpose

Performance testing evaluates how the **Automated Network Request Management** system behaves under expected and peak usage conditions. The goal is to confirm that the Service Catalog, Flow Designer approval workflow, CMDB validation checks, and IntegrationHub deployment calls all respond within acceptable time limits and remain stable as the number of concurrent requests grows well before the solution is rolled out organization-wide.

2. Scope

This document covers performance testing of the core request lifecycle: catalog browsing and submission, approval routing, CMDB validation, and automated network deployment triggering. It does not cover functional correctness or security/penetration testing.

3. Objectives

- Measure catalog item load time and form submission response time under normal and peak load.
- Measure the time taken for Flow Designer to trigger an approval task after a RITM is created.
- Measure CMDB topology lookup and validation response time.
- Measure IntegrationHub call latency when triggering Ansible/firewall configuration deployment.
- Determine the maximum number of concurrent network requests the instance can handle before performance degrades.
- Identify bottlenecks in the flow (approval routing, CMDB checks, or integration calls) that could delay fulfillment at scale.

4. Test Environment

Instance	ServiceNow Personal Developer Instance (PDI)
Modules	Service Catalog, Flow Designer, CMDB, IntegrationHub
Integration Endpoint	Simulated/mock Ansible or Firewall REST endpoint (for demo purposes)
Testing Tools	Apache JMeter (HTTP request simulation), ServiceNow Performance Analytics, browser DevTools Network tab
Test Data	Synthetic set of employee/manager test users and sample network request items

5. Performance Test Types

5.1 Load Testing

Simulates a normal expected number of concurrent users submitting network requests to verify the system meets response-time targets under typical daily usage.

5.2 Stress Testing

Gradually increases concurrent requests beyond expected levels to identify the breaking point e.g., the number of simultaneous catalog submissions or approval triggers before response times degrade significantly or errors occur.

5.3 Endurance (Soak) Testing

Runs a sustained moderate load over an extended period (e.g., several hours) to check for memory leaks, queue backlogs in Flow Designer, or degrading response times over time.

5.4 Spike Testing

Simulates a sudden burst of requests (e.g., during a major IT migration wave) to confirm the system recovers gracefully without dropping or duplicating requests.

6. Test Scenarios & Metrics

Scenario	Metric Measured	Target Threshold	Test Type
Catalog item page load	Page load time	<2 seconds	Load
Request form submission	Response time to create RITM	<3 seconds	Load
50 concurrent submissions	Average response time, error rate	<5 seconds, 0% errors	Load
200 concurrent submissions	Response time degradation point	Identify breaking point	Stress
Flow Designer approval trigger	Time from RITM creation to approval task creation	<10 seconds	Load
CMDB validation check	Query/validation response time	<3 seconds	Load
IntegrationHub deployment call	API call latency to mock endpoint	<5 seconds	Load
Sustained 6-hour moderate load	Response time drift, queue backlog	No significant drift	Endurance
Sudden burst (100 requests / 1 min)	Error rate, recovery time	0% dropped requests	Spike

7. Test Approach

- Define baseline single-user response times for each key transaction (catalog load, submission, approval trigger, CMDB check, deployment call).
- Use Apache JMeter to script concurrent virtual users hitting the Service Portal and simulate simultaneous catalog submissions.
- Gradually increase concurrent users (10 → 50 → 100 → 200) to trace how response time and error rate scale.
- Monitor ServiceNow Performance Analytics and system logs during each run to capture server-side processing time, not just network round-trip time.
- Run an endurance test with a steady moderate load (e.g., 20 concurrent users) for 4-6 hours to detect any gradual degradation.
- Record results in the results table below and compare them against target thresholds to identify pass/fail status.

8. Results Template

Scenario	Users / Load	Avg. Response Time	Pass / Fail
Catalog page load	1 User		
Request submission	1 User		
50 concurrent submissions	50 Users		
200 concurrent submissions	200 Users		

Scenario	Users / Load	Avg. Response Time	Pass / Fail
Approval trigger	Standard Load		
CMDB validation check	Standard Load		
IntegrationHub call	Standard Load		
Endurance (6 hr)	20 Users		

9. Risks & Mitigations

Risk	Mitigation
PDI resource limits skew results vs. production	Document PDI constraints and treat results as relative/directional, not absolute production benchmarks.
Mock Ansible/firewall endpoint doesn't reflect real API latency	Add a configurable artificial delay to the mock endpoint to approximate realistic external API response times.
Flow Designer approval queue backlog under high load	Test with tiered approval and timeout escalation enabled to ensure queues don't grow unbounded.
Topology update race conditions under concurrent requests	Verify CMDB correctly updates CIs so no two concurrent requests conflict on the same network segment.

10. Acceptance Criteria

- 95% of catalog page loads and request submissions complete within their target thresholds under expected load.
- No more than 1% error rate at 50 concurrent users; system does not crash or lock up at 200 concurrent users, even if response time increases.
- No unbounded growth in approval queue or response time drift during the 6-hour endurance run.
- Topology updates remain accurate (no data corruption) under concurrent request scenarios.
- System recovers to baseline response times within 5 minutes after a load spike ends.